Lab 33 — SIEM Alerting and Monitoring with Wazuh

> **SY0-701 Objective 4.4** — Explain security alerting and monitoring concepts and tools.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~25 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Run a single-node Wazuh SIEM, aggregate an extra application log, write a custom detection, trigger and read real alerts, report on them, tune out a known-good noisy source, query the REST API and switch on archiving.

You'll build: A working Wazuh manager with a custom log source, two local rules (one detection and one tuning suppression), an alert summary report and JSON archiving enabled.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; start the longest session available, because the install takes 5 to 10 minutes and needs about 4 GB of RAM; Lab 35 (SSH) helps but is not required.

Background

- A **SIEM** (security information and event management) performs **log aggregation** from systems, applications and infrastructure, normalises events, **correlates** them and raises **alerts**. It also supports **reporting** and **archiving** for investigations and compliance.
- **Agents** run on endpoints and forward data securely. **Agentless** collection pulls or receives data over syslog, APIs or SSH. A Wazuh manager also monitors its own logs, so this lab needs no separate agent.
- **Alert response and remediation** starts with triage: is it a true positive? Then **quarantine** or contain, remediate and validate.
- **Alert tuning** narrowly suppresses known-good activity (such as an approved vulnerability scanner) so that real alerts are not lost in noise. Tune by exception and never by disabling the whole rule.
- **Archiving** keeps raw events for the retention period, so you can hunt retrospectively when a new indicator appears.

Steps

1. Install the Wazuh all-in-one stack

The official installer deploys the manager, indexer and dashboard on one host. `-i` skips the hardware check on small lab VMs.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y curl jq openssh-client
cd /root && curl -sO https://packages.wazuh.com/4.9/wazuh-install.sh
bash wazuh-install.sh -a -i 2>&1 | tail -8
```

Note the dashboard `admin` password printed at the end. All generated passwords are also in `/root/wazuh-install-files.tar`.

2. Confirm the services and collect the API password

Check the three components and their ports: 1514 receives agent events, 55000 is the REST API and 9200 is the indexer.

```
for s in wazuh-manager wazuh-indexer wazuh-dashboard; do printf '%-16s %s\n' "$s"
"$(systemctl is-active $s)"; done
ss -tlnp | grep -E ':(1514|1515|55000|9200|443) '
API_PASS=$(tar -xOf /root/wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
| grep -A1 "api_username: 'wazuh'" | awk -F"'" '/api_password/ {print $2}')
echo "API password captured: ${API_PASS:+yes}"
```

3. Aggregate an application log and add a custom detection

Tell the manager to collect a new application log, then write a local rule that alerts when the app records a bulk export of customer data. Restart the manager to load both changes and test the rule offline with `wazuh-logtest`.

```
touch /var/log/labapp.log
cat >> /var/ossec/etc/ossec.conf <<'EOF'
<ossec_config>
  <localfile>
    <log_format>syslog</log_format>
    <location>/var/log/labapp.log</location>
  </localfile>
</ossec_config>
EOF
cat >> /var/ossec/etc/rules/local_rules.xml <<'EOF'

<group name="local,labapp,">
  <rule id="100100" level="10">
    <match>LABAPP: bulk export</match>
    <description>Lab app: bulk export of customer records (possible data exfiltration)
</description>
    <mitre><id>T1567</id></mitre>
  </rule>
</group>
EOF
systemctl restart wazuh-manager && sleep 10
echo 'Oct 28 10:15:00 web01 labapp: LABAPP: bulk export user=bob rows=50000' |
/var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '100100'|level:|description:"
```

4. Generate security events

Create two kinds of evidence: repeated SSH logins as non-existent users (Wazuh rules 5710 and the 5712 brute-force correlation) and a bulk export written to the application log.

```
for i in $(seq 1 12); do ssh -o BatchMode=yes -o StrictHostKeyChecking=no -o
ConnectTimeout=3 "baduser$i@127.0.0.1" true 2>/dev/null; done
echo "$(date '+%b %e %H:%M:%S') $(hostname) labapp: LABAPP: bulk export user=bob
rows=50000" >> /var/log/labapp.log
sleep 20
```

5. Read the alerts and build a report

Alerts land in `/var/ossec/logs/alerts/alerts.json`. Read the relevant ones, then produce a simple count by rule, which is the start of a SOC report.

```
ALERTS=/var/ossec/logs/alerts/alerts.json
jq -c 'select(.rule.id=="5710" or .rule.id=="5712" or .rule.id=="100100") |
{time:.timestamp, rule:.rule.id, level:.rule.level, desc:.rule.description,
srcip:.data.srcip}' "$ALERTS" | tail -6
echo "== Alert summary by rule =="
jq -r '"\(.rule.level)\t\(.rule.id)\t\(.rule.description)"' "$ALERTS" | sort | uniq -c |
sort -nr | head -10 | tee /root/alert-summary.txt
```

6. Tune out a known-good source

An approved vulnerability scanner at `10.99.99.99` triggers invalid-user alerts every night. Add a narrowly scoped child rule at level 0 that silences only that source, then prove it with `wazuh-logtest`: the same event from another address still alerts.

```
cat >> /var/ossec/etc/rules/local_rules.xml <<'EOF'

<group name="local,tuning,">
  <rule id="100110" level="0">
    <if_sid>5710</if_sid>
    <srcip>10.99.99.99</srcip>
    <description>Tuning: approved scanner 10.99.99.99 (change CR-2026-118) -
suppressed</description>
  </rule>
</group>
EOF
systemctl restart wazuh-manager && sleep 10
printf '%s\n' "Oct 28 02:00:01 web01 sshd[4242]: Invalid user admin from 10.99.99.99 port
51000" \
  | /var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '|level:"
printf '%s\n' "Oct 28 02:00:02 web01 sshd[4243]: Invalid user admin from 203.0.113.50 port
51001" \
  | /var/ossec/bin/wazuh-logtest 2>&1 | grep -E "id: '|level:"
```

The first event matches rule `100110` at level `0` (no alert). The second matches `5710` at level `5` (alert).

7. Query the SIEM through its REST API

SOAR playbooks and reporting tools talk to the SIEM through its API. Authenticate, then read the manager status and the agent list.

```
API_PASS=$(tar -xOf /root/wazuh-install-files.tar wazuh-install-files/wazuh-passwords.txt
| grep -A1 "api_username: 'wazuh'" | awk -F"'" '/api_password/ {print $2}')
TOKEN=$(curl -sk -u "wazuh:$API_PASS" -X POST
"https://localhost:55000/security/user/authenticate?raw=true")
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost:55000/manager/status" | jq
'.data.affected_items[0]' | head -12
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost:55000/agents?
select=name,ip,status" | jq '.data.affected_items'
```

8. Turn on archiving and check retention

By default only alerts are stored. Enable the JSON archive so every event is kept for retrospective hunting, and look at how Wazuh rotates and compresses logs by date.

```
sed -i 's|<logall_json>no</logall_json>|<logall_json>yes</logall_json>|'
/var/ossec/etc/ossec.conf
grep -m1 logall_json /var/ossec/etc/ossec.conf
systemctl restart wazuh-manager && sleep 15
echo "$(date '+%b %e %H:%M:%S') $(hostname) labapp: LABAPP: login user=alice" >>
/var/log/labapp.log
sleep 10
tail -1 /var/ossec/logs/archives/archives.json | jq -c '{time:.timestamp,
location:.location, log:.full_log}'
ls /var/ossec/logs/alerts/ /var/ossec/logs/archives/
```

Test it

```
jq -r 'select(.rule.id=="100100" or .rule.id=="5712" or .rule.id=="5710") | .rule.id'
/var/ossec/logs/alerts/alerts.json | sort | uniq -c
grep -c '"rule":{"level":0' /var/ossec/logs/alerts/alerts.json || true
test -s /var/ossec/logs/archives/archives.json && echo "ARCHIVING ON"
```

Expected: counts for `5710` and `100100` (and `5712` once enough attempts fall inside its time window), no level-0 alerts in the alert file, and `ARCHIVING ON`.

Troubleshooting

Symptom	Fix
The installer stops with a memory or CPU error	Keep the `-i` flag. If the indexer is killed for lack of memory, restart it with `systemctl restart wazuh-indexer` and run `bash wazuh-install.sh -a -i -o` to overwrite the partial install.
No 5710 alerts appear	Check that sshd logs go where Wazuh reads them: `grep -n 'journald|auth.log' /var/ossec/etc/ossec.conf`, and run `journalctl -u ssh -n 5`.
`wazuh-manager` fails after editing rules	A typo in the XML. Run `/var/ossec/bin/wazuh-analysisd -t` to see the line, fix it, then restart.

Challenge

Write a correlation rule (`frequency` and `timeframe` with `if_matched_sid`) that raises a level 12 alert when the same user triggers rule 100100 three times within ten minutes, and map it to the correct MITRE ATT&CK technique.

Reflection

Your SOC receives 4,000 alerts a day and analysts have started ignoring them. Which 4.4 activities and alert-response techniques do you apply first, and how do you prove that tuning did not create a false negative?

Exam tips

- Log aggregation collects; correlation links events; alerting notifies; archiving retains. Expect a scenario asking which activity is missing.
- Alert tuning reduces false positives. Over-tuning creates false negatives, which are worse.
- Agent-based collection gives deep host visibility; agentless (syslog, API, SNMP) suits appliances that cannot run software.
- A SIEM centralises and correlates; SOAR automates the response. Do not confuse the two.

References

- Wazuh documentation — https://documentation.wazuh.com
- Wazuh ruleset and custom rules — https://documentation.wazuh.com/current/user-manual/ruleset/index.html
- NIST SP 800-92, Guide to Computer Security Log Management — https://csrc.nist.gov/pubs/sp/800/92/final
- Sigma generic detection rules — https://github.com/SigmaHQ/sigma
- MITRE ATT&CK — https://attack.mitre.org
